



ChatAI.

An AI Chatbot Built by Students, for Students.

David Bahidika

🔧 API Integration

Aleshia McRae

🧠 Conversation Logic

Isaiah Alfred

🎨 UI & Testing

System Requirements

01

Async / Await

Every API call is non-blocking. The app stays fast and responsive — never freezes while waiting for GPT.

Requirement: Responsiveness

02

Polly Retry Logic

If the API fails, it waits and tries again automatically. 2s → 4s → 8s.
No crash, ever.

Requirement: Reliability

03

Conversation Manager

Full memory. Multiple sessions. Save and reload conversations across app restarts.

Requirement: Memory

04

Sentiment Analysis

Every message is classified as Positive, Neutral, or Negative in real time.

Requirement: Intelligence



Four layers. Each with one job.



Presentation

Layer

`ConsoleUI.cs`

Isaiah

What the user sees and types into. Never touches data directly.



Service

Layer

`OpenAIService`
`ConversationManager`
`SentimentService`

David + Aleshia

All the business logic. Where the real work happens.



Model

Layer

`Message.cs`
`Conversation.cs`
`OpenAIOptions.cs`

Shared

Data structures. Defines what a message and conversation look like.



Infrastructure

Layer

`Serilog`
`DI Container`
`appsettings.json`

Shared

Logging, config, and dependency wiring at startup.



David Bahidika.

The connection between the app and OpenAI.



OpenAIService.cs

The class that calls GPT. Implements IOpenAIService interface.



async / await

Non-blocking calls. App never freezes waiting for GPT.



Polly Exponential Backoff

Fails gracefully. Retries at 2s → 4s → 8s intervals.



Dependency Injection

All services auto-wired at startup in Program.cs.

Project Foundation & Structure Setup

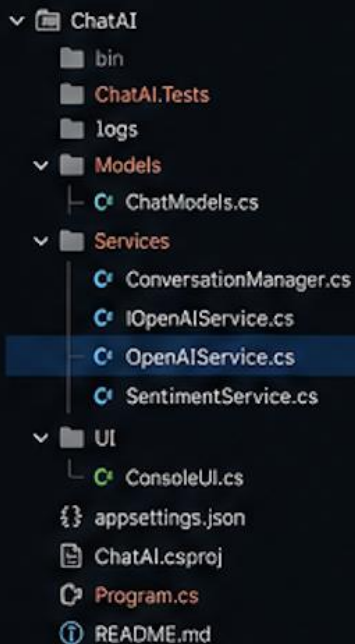
The project was initiated by setting up the complete structure and essential files, creating a solid and organized foundation for the entire system.



What this achieved:

- ✓ Consistent project structure
- ✓ Easy collaboration for all members
- ✓ Clean separation of concerns
- ✓ Smooth integration of features

Solution Structure



Models

Data structures used across the application



Services

Core business logic and API integration



UI

User interaction and presentation



Infrastructure

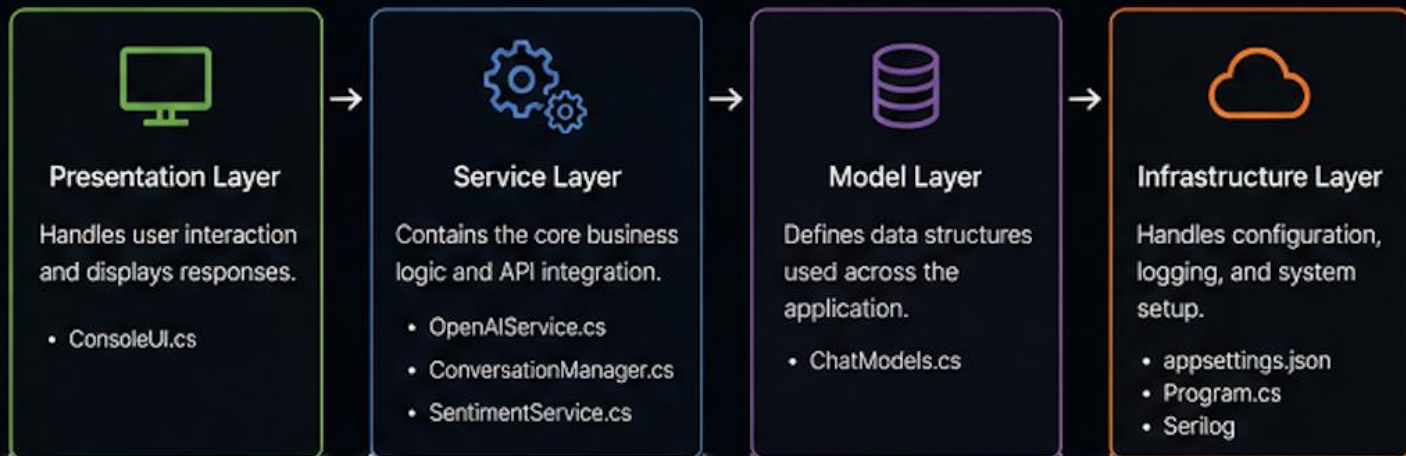
Configuration, logging, and system setup



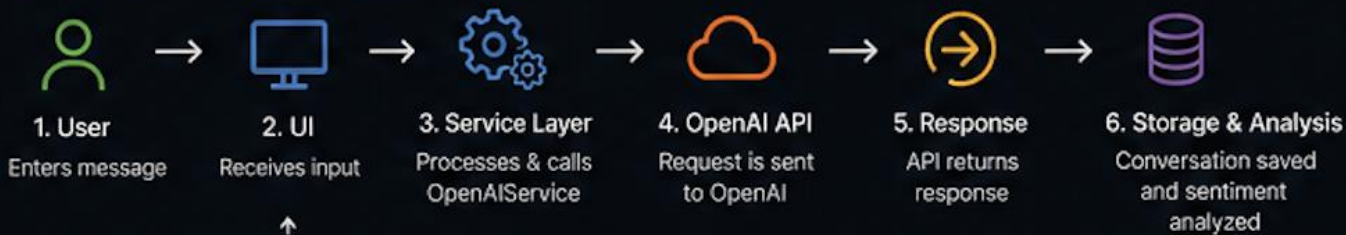
This structure allowed every feature to be added in the right place, keeping the system maintainable and scalable.

System Architecture & Workflow

A layered architecture ensures separation of concerns and smooth flow of data.



System Workflow (Block Diagram)



Core Contribution – API Integration & Impact

The core service layer is responsible for communicating with the OpenAI API. It is built to be fast, reliable, and resilient.

Key Features Implemented

- ✓ Asynchronous API calls (async/await)
- ✓ Polly retry logic for reliability
- ✓ Handles requests and responses
- ✓ Proper error handling and logging

```
public async Task<string> GetChatResponseAsync(List<ChatMessage> messages)
{
    var policy = Policy
        .Handle<HttpRequestException>()
        .WaitAndRetryAsync(3,
            retryAttempt => TimeSpan.FromSeconds(
                Math.Pow(2, retryAttempt)));

    var response = await policy.ExecuteAsync(async () =>
    {
        var request = CreateRequest(messages);
        var httpResponse = await _http.PostAsJsonAsync(_url, request);
        response.EnsureSuccessStatusCode();
        var result = await response.Content.ReadAsStringAsync();
        return ParseResponse(result);
    });
    return response;
}
```

Impact on the System



Responsive

Async/await keeps the application fast and non-blocking



Reliable

Polly retry logic handles failures gracefully



Maintainable

Clean architecture makes the code easy to maintain



Production-Ready

Logging, configuration, and error handling make it robust



The foundation and integration layer are the reason the entire AI chatbot system works smoothly and reliably.



```
public class OpenAIService : IOpenAIService
{
    private readonly HttpClient _http;
    private readonly AsyncRetryPolicy _retry;

    public OpenAIService(HttpClient http)
    {
        _http = http;

        // 🔄 Polly: retry 3x with exponential backoff
        _retry = Policy
            .Handle<HttpRequestException>()
            .WaitAndRetryAsync(3, attempt =>
                TimeSpan.FromSeconds(Math.Pow(2, attempt)));
        // attempt 1 = 2s | attempt 2 = 4s | attempt 3 = 8s
    }

    public async Task<string> SendMessageAsync(string message)
    {
        return await _retry.ExecuteAsync(async () =>
        {
            // ⚡ async call - thread never blocks
            var response = await _http.PostAsJsonAsync(
                "/v1/chat/completions",
                new { model = "gpt-4",
                    messages = new[] {
                        new { role = "user", content = message }
                    }
                });

            response.EnsureSuccessStatusCode();
            return await ParseResponse(response);
        });
    }
}
```


Challenges

**01**

API Key Security

Can't hardcode the key visible on GitHub.
Used the Options Pattern: app reads from environment variable first, falls back to appsettings.json. Key never committed to code.

**02**

End-to-End Async

Every method in the chain must be async.
One blocking .Result or .Wait() call creates a deadlock — the whole app hangs. Traced every layer until the chain was clean.

**03**

Polly Timing

First version retried too fast and immediately hit the rate limit again. Tested different intervals — settled on 2s, 4s, 8s. Balance between speed and not spamming the server.



Aleshia McRae.

Building the app's memory.



Conversation Manager.

Created ConversationManager.cs to maintain chat flow and state data transitions



Sentiment Service

Integrated real time tone detection to categorize user messages by emotional intent.



JSON Persistence

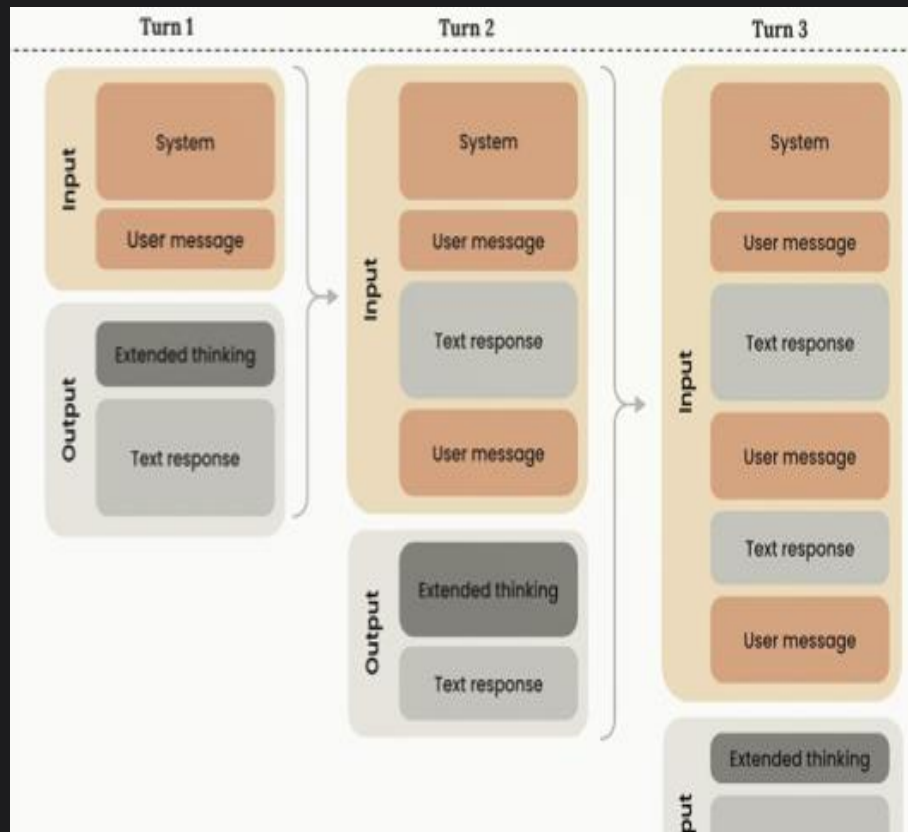
Implemented JSON based storage for loading and saving multi session dialogue history.



Strategy

LLMs are naturally stateless, the logic used manages a context window to feed relevant history back into every API call.

- ✓ Message Capture: Records Q&A pairs in a thread style list.
- ✓ Assembly: Wraps user input with preceding context.
- ✓ Optimization: Prunes history to stay within API token limits.



Architecture



ConversationManager.cs

The ConversationManager.cs performs a silent cycle every time the user hits enter. It retrieves the last 10 messages from memory to maintain context.



This prevents any "Reference Failure" errors, ensuring AI knows what "that" refers to in follow questions.



0 references

```
38 public void AddMessage(string role, string content)
39 {
40     var message = new Message(role, content);
41
42
43     if (role.ToLower() == "user")
44     {
45         message.Sentiment = _sentimentService.Analyze(content);
46     }
47     else
48     {
49         message.Sentiment = "Neutral";
50     }
51
52     _activeConversation.Messages.Add(message);
53 }
54
```

Analysis



SentimentServices.cs

Each users message is analyzed for the tone and intent before the API call. It implements empathy; if frustrated its detected.



- + Positive: Detects satisfaction and reinforces helpfulness.
- Negative: Triggers empathetic response modifiers.
- = Neutral: Standard analytical processing.

ChatAi.sln ChatAi.csproj appsettings.json SentimentService.cs × ConversationManager.cs Program.cs

Services > SentimentService.cs > {} ChatAi.Services > SentimentService > Analyze(string) : string

```
1  using ChatAI.Models;
2
3  namespace ChatAI.Services
4  {
5      4 references
6      public interface ISentimentService
7      {
8          4 references
9          string Analyze(string text);
10
11      6 references
12      public class SentimentService : ISentimentService
13      {
14          4 references
15          public string Analyze(string text)
16          {
17              if (string.IsNullOrEmpty(text)) return "Neutral";
18
19              var input = text.ToLower();
20
21              // Simple keyword-based sentiment analysis (for demonstration purposes)
22              var positiveWords = new[] { "happy", "great", "excellent", "thanks", "helpful", "love", "good" };
23              var negativeWords = new[] { "bad", "error", "terrible", "angry", "broken", "fail", "hate" };
24
25              int score = 0;
26              foreach (var word in positiveWords) if (input.Contains(word)) score++;
27              foreach (var word in negativeWords) if (input.Contains(word)) score--;
28
29              return score > 0 ? "Positive" : score < 0 ? "Negative" : "Neutral";
30          }
31      }
32  }
```

Persistence Layer



JSON Data

Implemented JSON serialization to ensure session are recoverable after the application restarts.



Save: System.Text.Json Serialization to .json files

Load: Async Deserialization into Memory Lists

Clear: Wipe current session while keeping archive.


```
59
60     3 references
61     public void SaveToFile(string filePath)
62     {
63         var options = new JsonSerializerOptions { WriteIndented = true };
64         string json = JsonSerializer.Serialize(_activeConversation, options);
65         File.WriteAllText(filePath, json);
66     }
67
68     3 references
69     public void LoadFromFile(string filePath)
70     {
71         if (File.Exists(filePath))
72         {
73             string json = File.ReadAllText(filePath);
74             var loaded = JsonSerializer.Deserialize<Conversation>(json);
75             if (loaded != null)
76             {
77                 _activeConversation = loaded;
78             }
79         }
80     }
81 }
```

Challenges

**01**

State Management

Multiple sessions in memory — wrong one kept becoming active. Fixed with encapsulation: `_active` is private, only changeable through `SwitchConversation()`. No direct field access allowed.

**02**

JSON Edge Cases

Special characters in messages corrupted the file format. Empty files crashed the load. Wrapped deserialization in try/catch — Serilog logs the error, app starts fresh instead of crashing.

**03**

Sentiment Accuracy

Short messages and sarcasm are nearly impossible to classify with keyword logic. Tested hundreds of messages, tuned the lists, added a neutral fallback for anything ambiguous.



Isaiah Alfred.

Building the UI.



ConsoleUI.cs

Presentation Layer. Reads input, displays output. Never touches data directly — Separation of Concerns.



Command Pattern

/new /list /history /save /load /search /stats /help — each validated before executing.



xUnit + Moq

Full test suite. Moq fakes the OpenAI API — tests run free, fast, and without real calls.



Documentation

README, inline comments, architecture notes — kept current throughout the whole project.

Interaction!



ConsoleUI.cs

The `ConsoleUI.cs` runs a continuous interaction loop each time the user presses enter. It captures user input, checks for commands, and sends messages to the AI service, then displays the response back to the user in real time.

t.

```
Console.WriteLine("Chat AI started. Type /help for commands.");

while (true)
{
    Console.Write("> ");
    var input = Console.ReadLine();

    if (string.IsNullOrEmpty(input))
        continue;

    if (HandleCommand(input))
        continue;

    _manager.AddMessage("user", input);
    try
    {
        var response = await _chatService.SendMessageAsync(_manager.GetHistory());
        _manager.AddMessage("assistant", response);
        Console.WriteLine($"AI: {response}");
    }
    catch (Exception ex)
    {
    }
}
```

Challenges



01

ReturnsAsync vs Returns

.Returns() on an async mock compiles but crashes at runtime with `NullReferenceException`. Must use `.ReturnsAsync()` — it wraps the return value in a completed `Task` automatically.



02

Command Validation

Every command needs input checks before executing. `/load` with no filename, unknown commands, empty input — all need proper error messages. Used `IsNullOrWhiteSpace()` and array bounds checking throughout.



03

Proper Code Editing

Code editing created challenges that disrupted reliable ChatAI simulations. Issues like formatting errors, missing dependencies, or misconfigured settings in tools like Visual Studio Code often caused execution failures or inaccurate results.

Tools



INTEGRATION

OpenAI NuGet

GPT API client for .NET. Handles auth + request formatting.



RESILIENCE

Polly

Retry, circuit breaker, and timeout resilience policies.



LOGGING

Serilog

Structured rolling daily log files written to /logs.



TESTING

xUnit

Industry-standard test runner for .NET applications.



TESTING

Moq

Mocking library — fake interface implementations for tests.



ARCHITECTURE

MS Extensions DI

IoC container. Wires all services together at startup.



Watch the three layers
work together live.

github.com/davidbah2703/ChatAi

What You'll See



Real GPT response

Live via OpenAIService.cs



Sentiment labeled

On every message, real time



Async in action

Zero freezing, instant UI



Three layers live

UI → Logic → API working together

Thank You.



Team ChatAI · FAMU · CIS 4301 · Spring 2026

David Bahidika · Aleshia McRae · Isaiah Alfred

Dr. Sumarga K. Sah Tyagi

 github.com/davidbah2703/ChatAi